

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №3
по дисциплине «Построение и анализ алгоритмов»
ТЕМА: РАССТОЯНИЕ ЛЕВЕНШТЕЙНА.

Студент гр. 4345

Бабий Д.Н.

Преподаватель

Жангиров Т.Р.

Цель работы.

Изучить и реализовать алгоритм Вагнера-Фишера для поиска редакционного расстояния. Реализовать новую операцию удаления разных, подряд идущих символов.

Задание

Над строкой ε (будем считать строкой непрерывную последовательность из латинских букв) заданы следующие операции:

1. $\text{replace}(\varepsilon, a, b)$ – заменить символ a на символ b .
2. $\text{insert}(\varepsilon, a)$ – вставить в строку символ a (на любую позицию).
3. $\text{delete}(\varepsilon, b)$ – удалить из строки символ b .

Входные данные:

Первая строка входных данных содержит строку из строчных латинских букв. S

Вторая строка входных данных содержит строку из строчных латинских букв. T

Выходные данные: одно число – минимальная стоимость операций.

Sample Input:

pedestal

stien

Sample Output:

7

Для всех вариантов:

1) Предполагается, что для весов операций действует правило треугольника: если две последовательные операции можно заменить одной, то это не ухудшает общую цену.

2) При выполнении операций запрещается применять операции к символам, которые уже были получены в результате выполнения операций (т.е. строка преобразуется всегда только слева направо) — актуально для вариантов 3–6, 15 (упрощающее условие).

Вариант 5а.

5а. Добавляется 4-я операция со своей стоимостью: удаление двух последовательных разных символов.

Выполнение работы:

В ходе выполнения работы была разработана функция `levenstein_4_ops`, реализующая модифицированный алгоритм вычисления расстояния Левенштейна. В отличие от классического алгоритма, в данной реализации добавлена четвёртая операция — удаление двух подряд идущих различных символов с заданной стоимостью.

$$D(i, j) = \begin{cases} 0, & i=0, j=0 \\ i, & j=0, i>0 \\ \min(D(i, j-1)+1, D(i, j-2)+cost), & j=0, j>0 \\ \min \begin{cases} D(i, j-1)+1 \\ D(i-1, j)+1 \\ D(i-1, j-1)+diff(str1[j], str2[j]) \\ D(i, j-2)+cost, j \geq 2, str1[j-1] \neq str1[j-2] \end{cases}, & j>0, i>0 \end{cases}, \text{ где } diff —$$

функция проверки равенства символов (1 — если неравны, 0 — если равны).

В отличие от классической реализации, здесь используется оптимизация по памяти: вместо хранения всей матрицы используются только две строки — текущая и предыдущая. Это позволяет снизить пространственную сложность алгоритма до $O(N)$.

Сначала инициализируется первая строка, которая соответствует преобразованию префиксов исходной строки в пустую строку. Значения заполняются с учётом стандартного удаления символов, а также с учётом новой операции удаления двух различных подряд идущих символов, если это даёт меньшую стоимость.

Далее происходит основной цикл по символам второй строки. На каждой итерации вычисляется новая строка динамической таблицы. Для каждой ячейки рассматриваются три стандартные операции (добавить, удалить, изменить), потом проверяется возможность применения новой операции, если текущие два символа в исходной строке различны, то рассматривается переход через две позиции влево с добавлением заданной стоимости. Это позволяет «перепрыгнуть» сразу два символа и потенциально уменьшить итоговую стоимость преобразования.

Результатом работы функции является значение в последней ячейке текущей строки, которое соответствует минимальной стоимости преобразования исходной строки в целевую с учётом всех допустимых операций.

Оценка временной сложности алгоритма составляет $O(N \times M)$, так как каждая пара символов обрабатывается ровно один раз. Пространственная сложность равна $O(N)$, поскольку в памяти одновременно хранятся только две строки таблицы.

Исходный код программы смотри в Приложении 1.

№	Входные данные	Выходные данные	Таблица	Комментарий
1	abxyz xyz	1	# a b x y z # 0 1 1 2 2 3 x 1 1 2 1 2 2 y 2 2 2 2 1 2 z 3 3 3 3 2 1	Удаляются последние два символа за одну операцию.
2	abxyaa abxy	2	# a b x y a a # 0 1 1 2 2 3 4 a 1 0 1 1 2 2 3 b 2 1 0 1 1 2 3 x 3 2 1 0 1 1 2 y 4 3 2 1 0 1 2	Последние два символа неравны, следовательно удалить их за одну операцию не можем.
3	фрышгвр ыф	4	# ф р ы ш г в р ы # 0 1 1 2 2 3 3 4 4 4	Правильное запоолнение первой строки.
4	фвыфывф	7	# # 0 ф 1 в 2 ы 3 ф 4 ы 5 в 6	Первая строка пустая, поэтому 4 операция не имеет действия и происходит сложение.

			ф 7	
--	--	--	-----	--

Таблица 1 – тестирование работы программы.

Вывод.

Был изучен и реализован алгоритм Вагнера–Фишера для вычисления редакционного расстояния с произвольными стоимостями операций. В алгоритм была добавлена четвёртая операция — удаление двух последовательных разных символов с собственной стоимостью, а также реализована возможность задавать индивидуальные стоимости операций замены и удаления для отдельных символов.

ПРИЛОЖЕНИЕ 1

ИСХОДНЫЙ КОД ПРОГРАММЫ

```
def levenstein_4_ops(str_1, str_2, cost):
    n, m = len(str_1), len(str_2)

    current_row = [0] * (n + 1)

    print("Таблица:")
    print("    ", *["#"] + list(str_1))

    for j in range(1, n + 1):
        current_row[j] = current_row[j - 1] + 1
        print(f"first_row[{j}] -> удаление '{str_1[j-1]}': {current_row[j]}")

        if j >= 2 and str_1[j - 1] != str_1[j - 2]:
            new_val = current_row[j - 2] + cost
            if new_val < current_row[j]:
                print(
                    f"first_row[{j}] -> удаление пары "
                    f"'{str_1[j-2]}{str_1[j-1]}' за {cost}: {new_val}"
                )
            current_row[j] = min(current_row[j], new_val)

    print("# ", *current_row)
    print()

    for i in range(1, m + 1):
        previous_row = current_row
        current_row = [i] + [0] * n

        print(f"Обрабатываем символ '{str_2[i-1]}'")

        for j in range(1, n + 1):
            cost_change = 0 if str_1[j - 1] == str_2[i - 1] else 1

            delete_cost = previous_row[j] + 1
            insert_cost = current_row[j - 1] + 1
            replace_cost = previous_row[j - 1] + cost_change

            res = min(delete_cost, insert_cost, replace_cost)

            print(
                f"cell[{i}][{j}] ({str_1[j-1]} -> {str_2[i-1]}): "
                f"del={delete_cost}, ins={insert_cost}, "
                f"rep={replace_cost}"
            )

            if j >= 2 and str_1[j - 1] != str_1[j - 2]:
                pair_delete = current_row[j - 2] + cost
                print(
                    f"cell[{i}][{j}] pair_del "
                    f"'{str_1[j-2]}{str_1[j-1]}' = {pair_delete}"
                )
                res = min(res, pair_delete)
```



```

        current_row[j] = res
        print(f" -> min = {res}")

    print(str_2[i - 1], *current_row)
    print()

    return current_row[n]

s1 = input()
s2 = input()
print("Ответ:", levenstein_4_ops(s1, s2, 1))

```